

Kafka 보안 구성 적용기

From PLAINTEXT To SASL_SSL

금기현

kbzjung359@gmail.com

Kafka 보안 구성 배경

SASL 인증 구성

Kafka ACL 연동

추가 고려 사항

감사 로그 수집

보안 구성 마이그레이션

Kafka 보안 구성 배경

기존의 Kafka 운영 환경

- 많은 트래픽을 수용할 수 있는 큰 **공용 클러스터** 구성
 - 여러 서비스 부서가 공유해서 사용
- 클러스터는 PLAINTEXT 리스너로 구성
- 민감 데이터를 처리하는 경우, 별도의 부서 전용 클러스터를 구성하여 격리
 - 컴플라이언스는 충족하지만 **비용과 운영 부담 증가**
- 대내외적으로 강화되는 보안 요구 사항

Kafka 보안 구성 배경

PLAINTEXT는 한계가 있다

- 목표: 민감 데이터 수용이 가능한 공용 클러스터 구성
 - 네트워크 접근 통제 → 방화벽 ACL
 - 클라이언트 인증 → **SASL**
 - 리소스(토픽, 그룹, TXID) 접근 통제 → **Kafka ACL**
 - 통신 구간 암호화 → SSL(TLS)
 - 감사 로그 수집 → *kafka.authorizer.logger*

SASL 인증 구성

SASL 인증 방식 비교

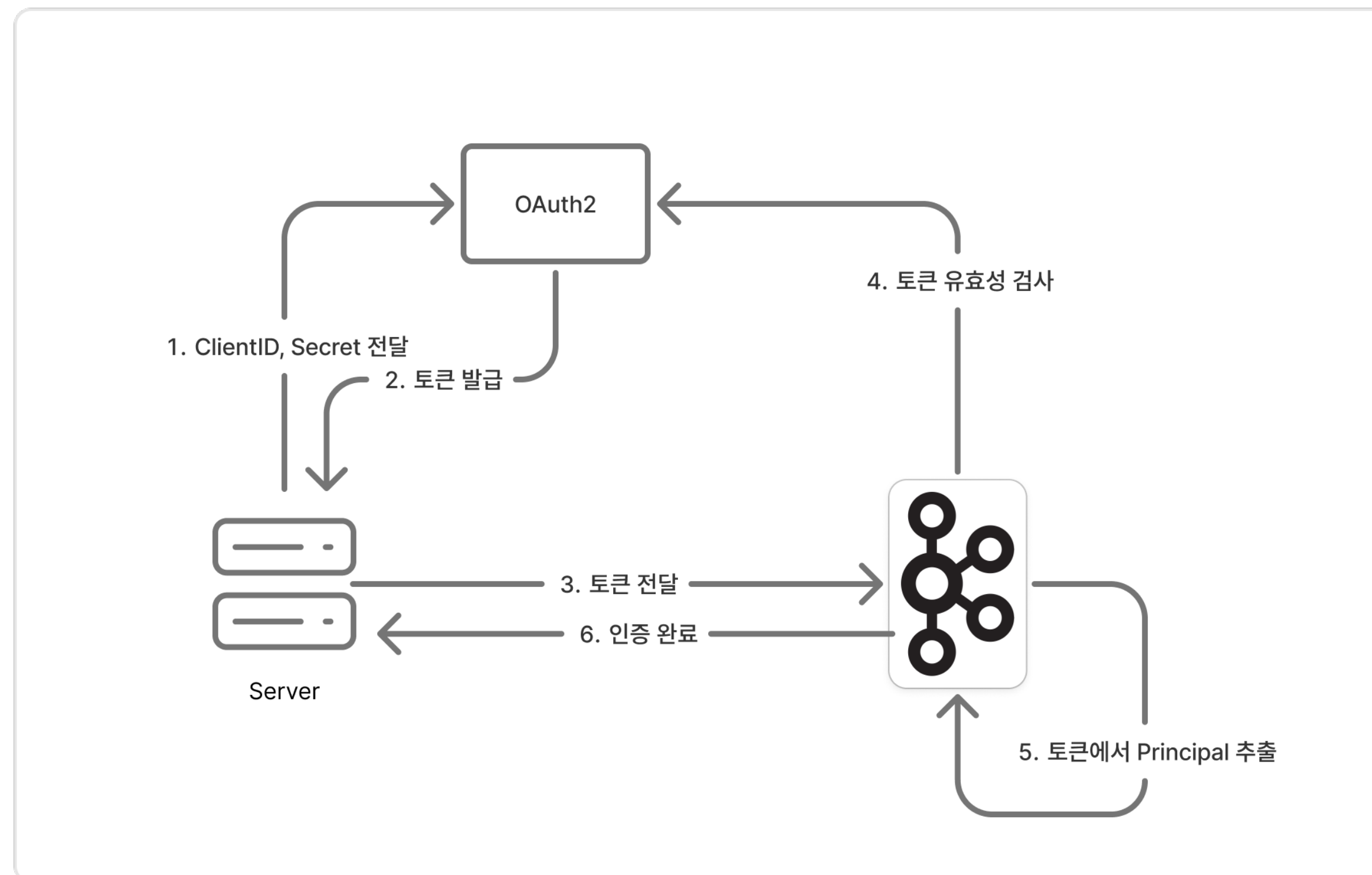
- Simple **A**uthentication and **S**ecurity **L**ayer
- Kafka는 크게 Kerberos, PLAIN, SCRAM, OAUTHBEARER 메커니즘 지원
 - Kerberos 는 관리 포인트가 추가되어 운영 부담으로 제외

	PLAIN	SCRAM	OAUTHBEARER
인증 방식	Username, Password 평문 전송	Challenge-Response	OAuth2 Access Token
Credential 저장	JAAS 설정 파일 or 외부 시스템	Kafka (ZK or KRaft)	OAuth2 인증 서버
클라이언트 지원 여부	O (대부분 지원)	O (대부분 지원)	△ (일부 미지원)
인증 로직 커스터마이징 여부	O (자유롭게 가능)	X (제한적)	△ (OAuth2 서버 필수, 일부 로직 수정 가능)
인증 시스템 장애 영향	△ (구현에 따라 다름)	X (Kafka 자체 처리)	O (Kafka 전면 장애)

SASL 인증 구성

SASL/OAUTHBEARER 검토

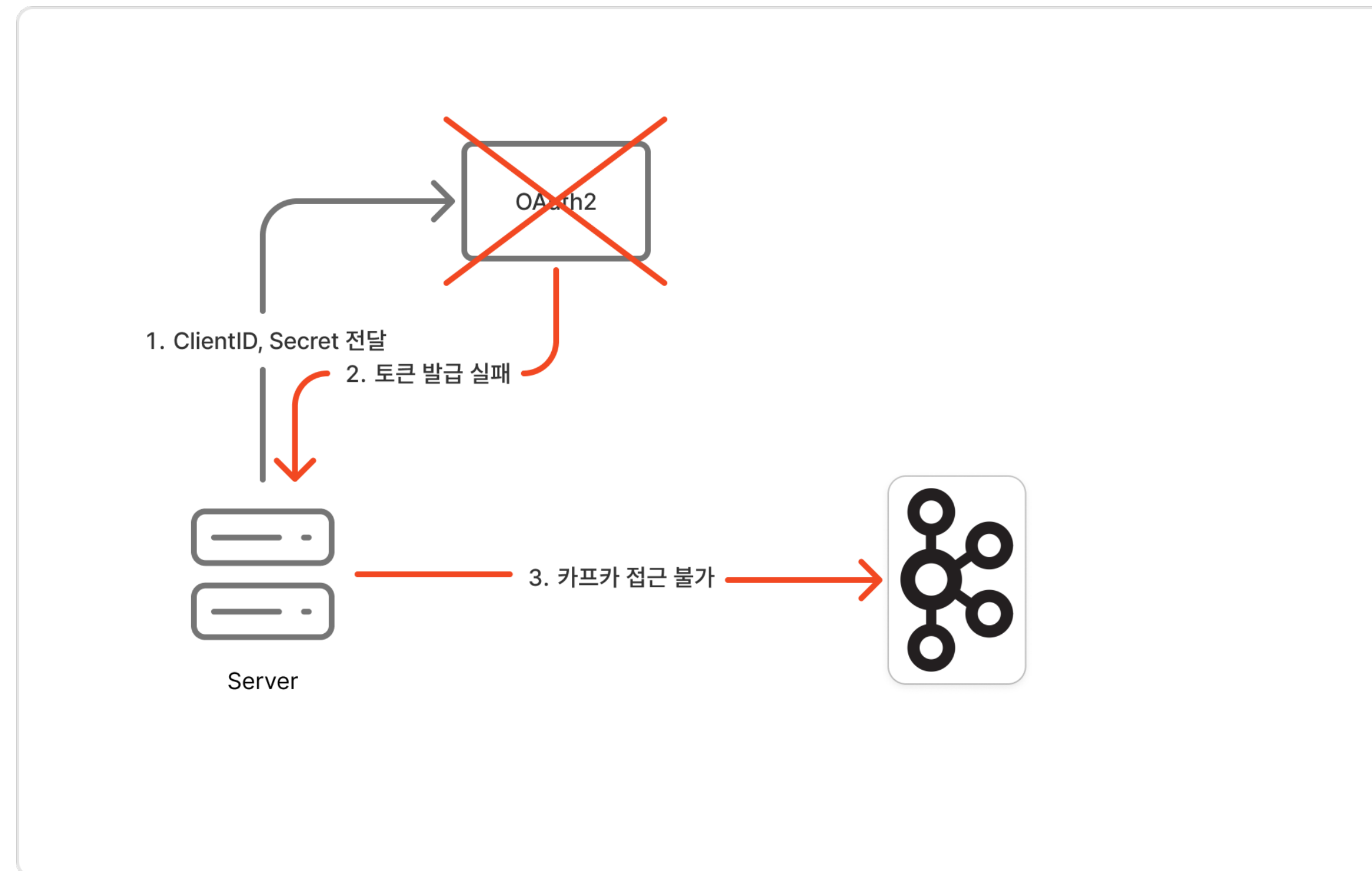
- 사내에 OAuth2 기반 인증 플랫폼 존재, 플랫폼 인증 구성시 사용 권고
- 인증 정보(Credential) 관리에 대한 부담도 해소 가능
- OAuth 서버 관리 부담이 없어 구성이 간편하고 동작 및 성능 테스트에서도 큰 이슈가 없었음



SASL 인증 구성

SASL/OAUTHBEARER 검토

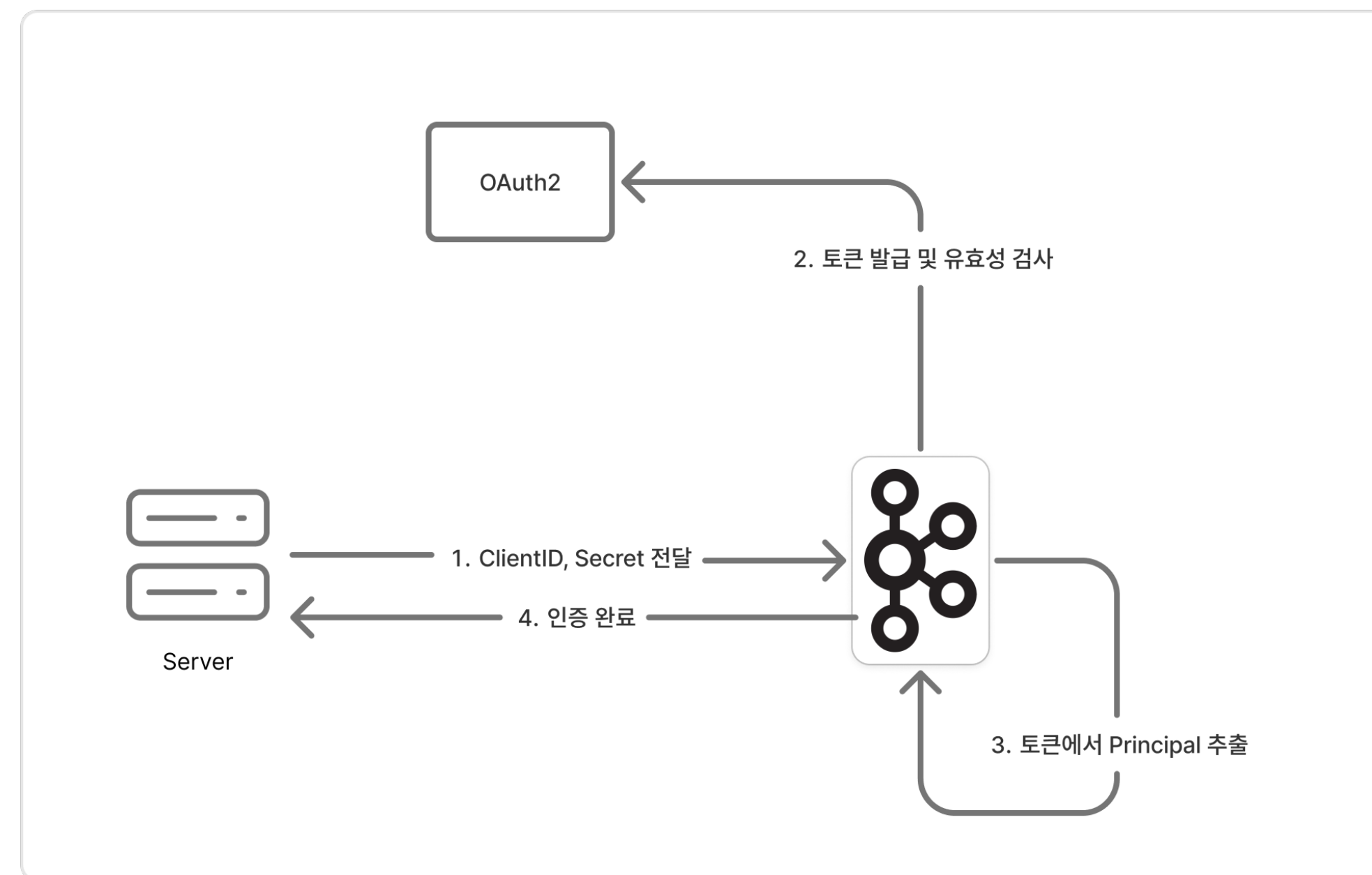
- 대답할 수 없는 질문들...
 - OAUTHBEARER 인증을 지원하는 클라이언트를 강제할 수 있을까?
 - OAuth2 인증 서버에 장애가 발생한다면?



SASL 인증 구성

SASL/PLAIN + 커스텀 플러그인 개발

- SASL/PLAIN 인증 방식에서 토큰 발급을 통한 인증을 수행하는 커스텀 플러그인 개발
 - **OAuth2 서버 장애를 감지하면 임시로 자체 인증** 수행하는 Failover 로직 추가
 - 사내 인증 플랫폼 사용 권고 & 외부 시스템 장애 내결함성 & 클라이언트 호환 모두 충족하는 구성 완성



- **⚠ SASL 인증은 TCP 커넥션 단위로 수행** → 외부 인증 시스템에 과도한 요청이 발생하지 않도록 주의 필요

Kafka ACL 연동

Kafka ACL 관리 방안

- Kafka는 플러그인 방식으로 인가 프레임워크 지원
 - *org.apache.kafka.server.authorizer.Authorizer*¹⁾ 인터페이스 구현 필요
 - ACL 생성/삭제/조회, ACL 평가에 대한 전반적인 로직 구현
- KRaft Metadata Log에 ACL을 저장하는 기본 구현체 제공
 - *org.apache.kafka.metadata.authorizer.StandardAuthorizer*²⁾
- 사내에 IAM과 유사한 플랫폼이 있었지만, 개편이 진행 중이었던 상황
 - 외부 의존성 없이 직접 Kafka ACL을 관리하도록 구성

1) <https://github.com/apache/kafka/blob/4.2.0/clients/src/main/java/org/apache/kafka/server/authorizer/Authorizer.java>

2) <https://github.com/apache/kafka/blob/4.2.0/metadata/src/main/java/org/apache/kafka/metadata/authorizer/StandardAuthorizer.java>

Kafka ACL 연동

서비스 부서에 부여할 Kafka ACL 정리

- 리소스 타입과 애플리케이션 구분 별 필요한 Kafka ACL¹⁾ 정리
- 토픽 생성/삭제와 같은 작업은 거버넌스 포탈을 통해 진행하도록 강제

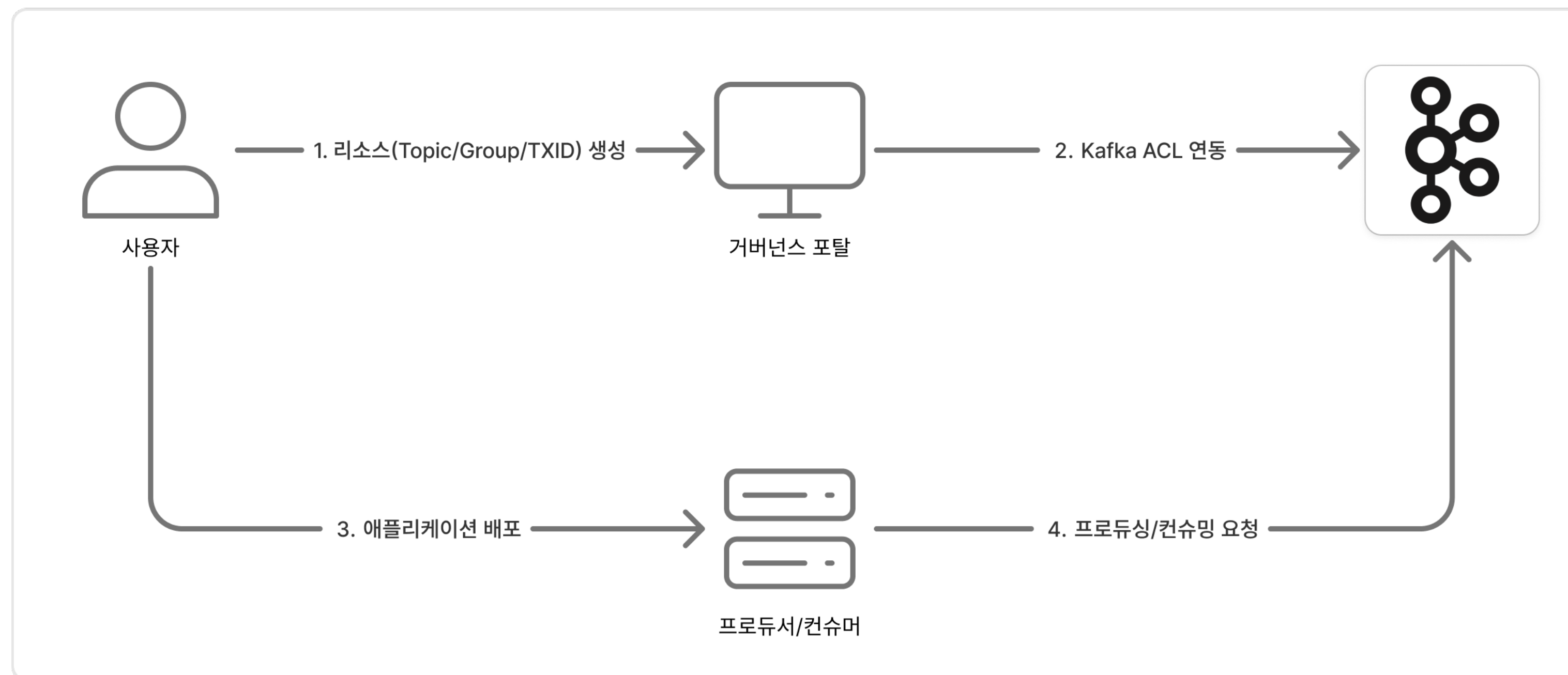
리소스 타입	애플리케이션 구분	Operations	사용자 단 추상화
Topic	Producer	Write, Describe, DescribeConfigs	TOPIC/WRITE
	Consumer	Read, Describe, DescribeConfigs	TOPIC/READ
Group	Consumer	Read, Describe	GROUP/READ
Transactional ID	Producer	Write, Describe	TXID/WRITE
Cluster	Producer	IdempotentWrite	전체 대상으로 허용, 관리X

1) <https://kafka.apache.org/42/security/authorization-and-acls/#operations-and-resources-on-protocols>

Kafka ACL 연동

Kafka ACL 거버넌스 포탈 연동

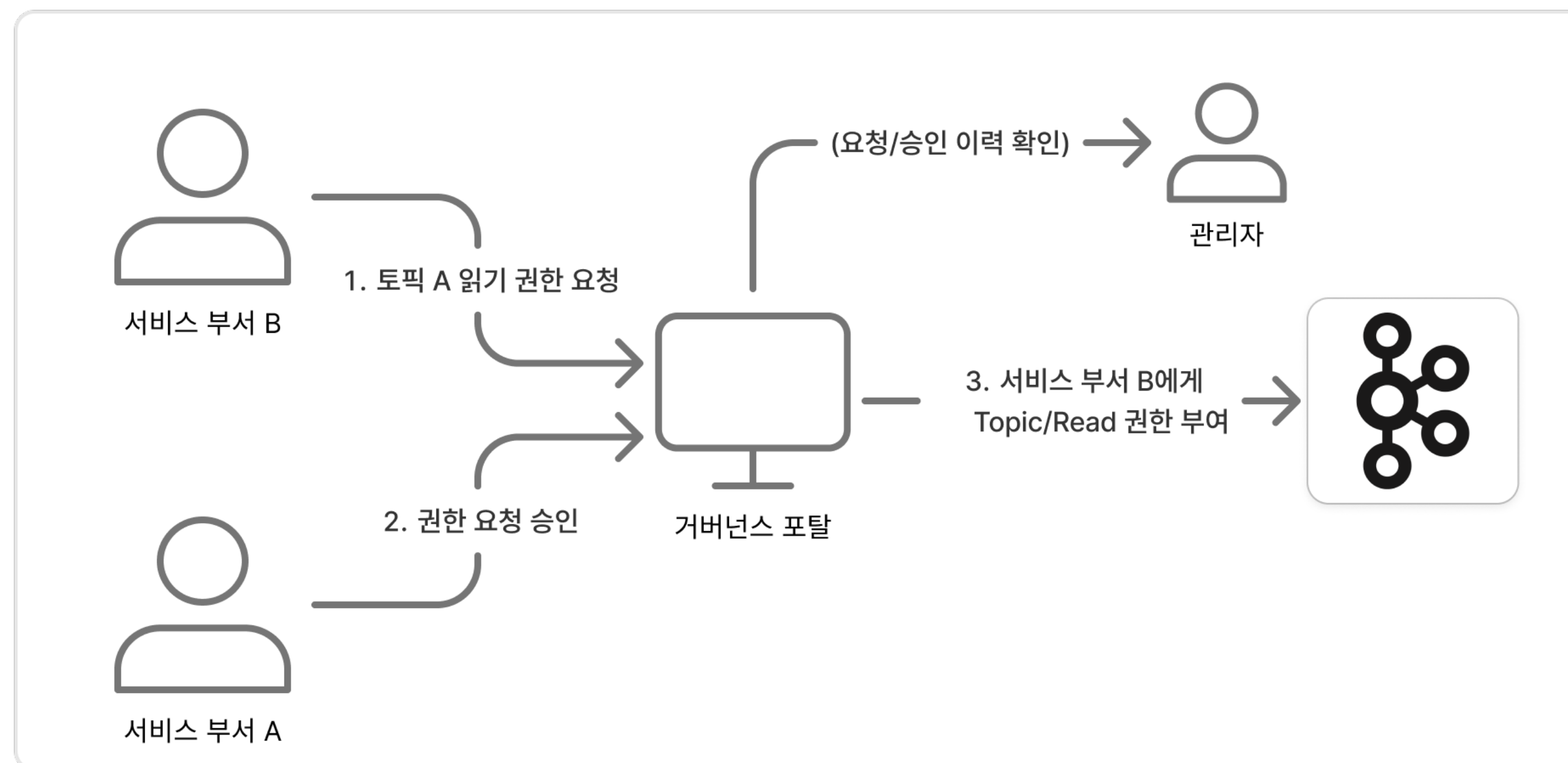
- 이미 토픽, 그룹에 대해 담당자와 부서 정보를 관리하는 거버넌스 포탈 운영중
- 거버넌스 포탈에 등록된 리소스 정보와 담당자 정보를 기반으로 Kafka ACL 연동 진행



Kafka ACL 연동

서비스 부서 간 리소스 공유

- 타 부서에서 토픽 접근이 필요한 경우, 거버넌스 포털을 통해 권한 요청 진행
 - 권한 요청에 대한 승인/거절은 토픽 담당 부서에서 진행
- 관리자는 권한 요청 이력만 확인



추가 고려 사항

감사 로그 수집

- 감사 로그는 kafka-authorizer.log 파일에 출력
- 기본 로깅 레벨은 INFO이며 접근이 거부된 요청에 한해 로깅
- 모든 요청에 대해 로깅이 필요하다면 **DEBUG**로 변경 필요
 - DEBUG로 변경하면 로그 용량이 급격하게 증가
 - 적절한 로그 삭제 및 외부 전송 정책 필요

```
log4j.appender.authorizerAppender=org.apache.log4j.DailyRollingFileAppender
```

```
log4j.appender.authorizerAppender.File=${kafka.logs.dir}/kafka-authorizer.log
```

```
# Access denials are logged at INFO level, change to DEBUG to also log allowed accesses
```

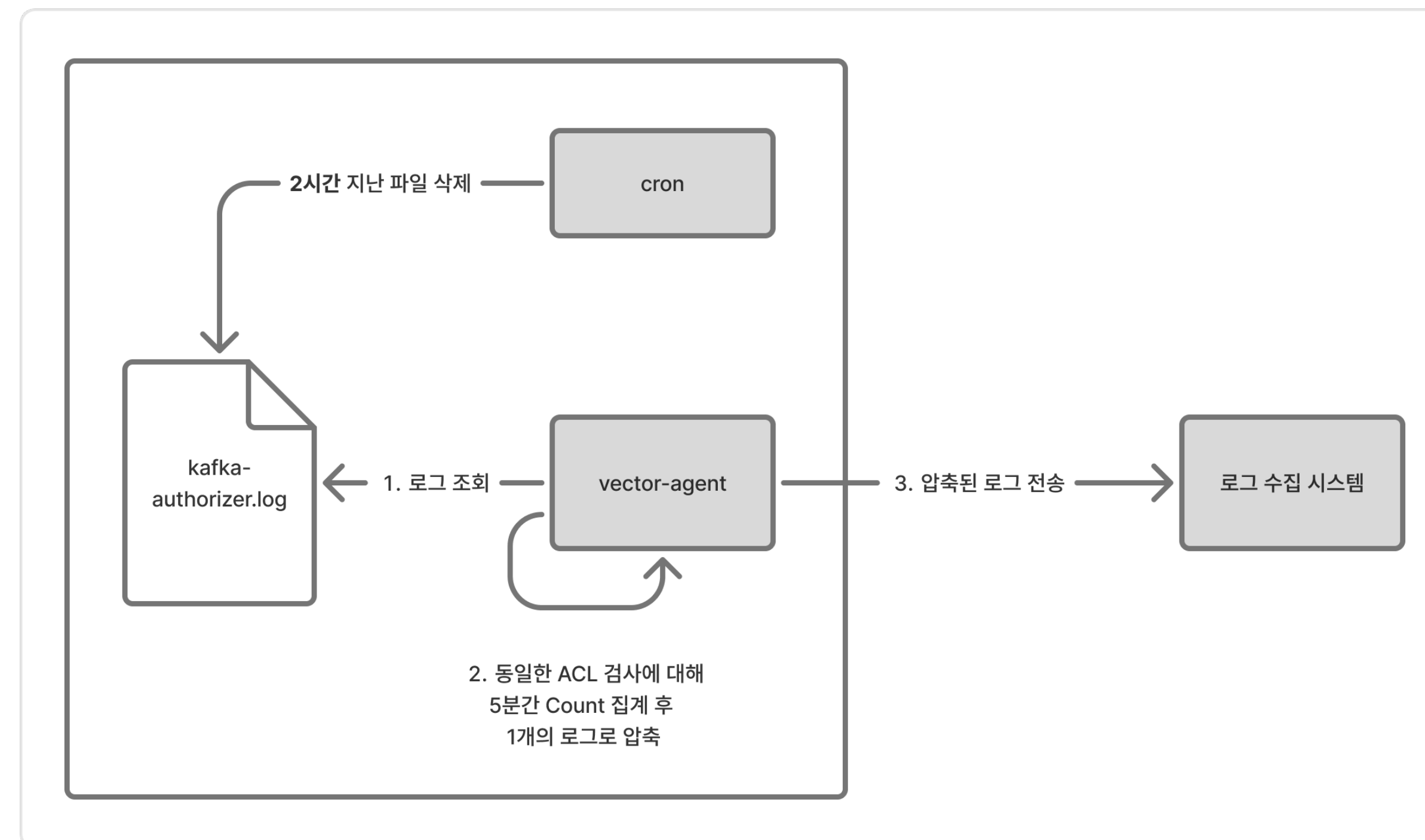
```
log4j.logger.kafka.authorizer.logger=DEBUG, authorizerAppender
```

```
log4j.additivity.kafka.authorizer.logger=false
```

추가 고려 사항

감사 로그 수집

- 목표 트래픽 생성 시 브로커 한 대당 50GB/h 로그 생성
 - 수 개월간 보관해야 하는 감사 로그 특성상 원본 그대로 수집하고 저장하는 것은 불가능
- Vector¹⁾ 로그 수집 에이전트에서 로그 압축 후 전송하도록 구성



1) <https://vector.dev/>

추가 고려 사항

PLAINTEXT → SASL_SSL 무중단 마이그레이션

- PLAINTEXT로 운영중인 클러스터에 대해 중단 없이 SASL_SSL로 마이그레이션 지원
- Dynamic Controller¹⁾ 활성화된 클러스터 -> v3.9 이상
 - 컨트롤러 멤버 구성을 동적으로 관리 가능 (컨트롤러 추가/제거/리스너 변경 등)
 - kafka-features.sh에서 kraft.version 피처를 통해 확인 가능
 - v4.1 에서 Static -> Dynamic Controller 피처 업그레이드 지원²⁾

```
> kafka-features --bootstrap-server ${V3_8_BOOTSTRAP_SERVER} describe
Feature: metadata.version      SupportedMinVersion: UNKNOWN 1 SupportedMaxVersion: 3.8-IV0      FinalizedVersionLevel: 3.8-IV0 Epoch: 108682570

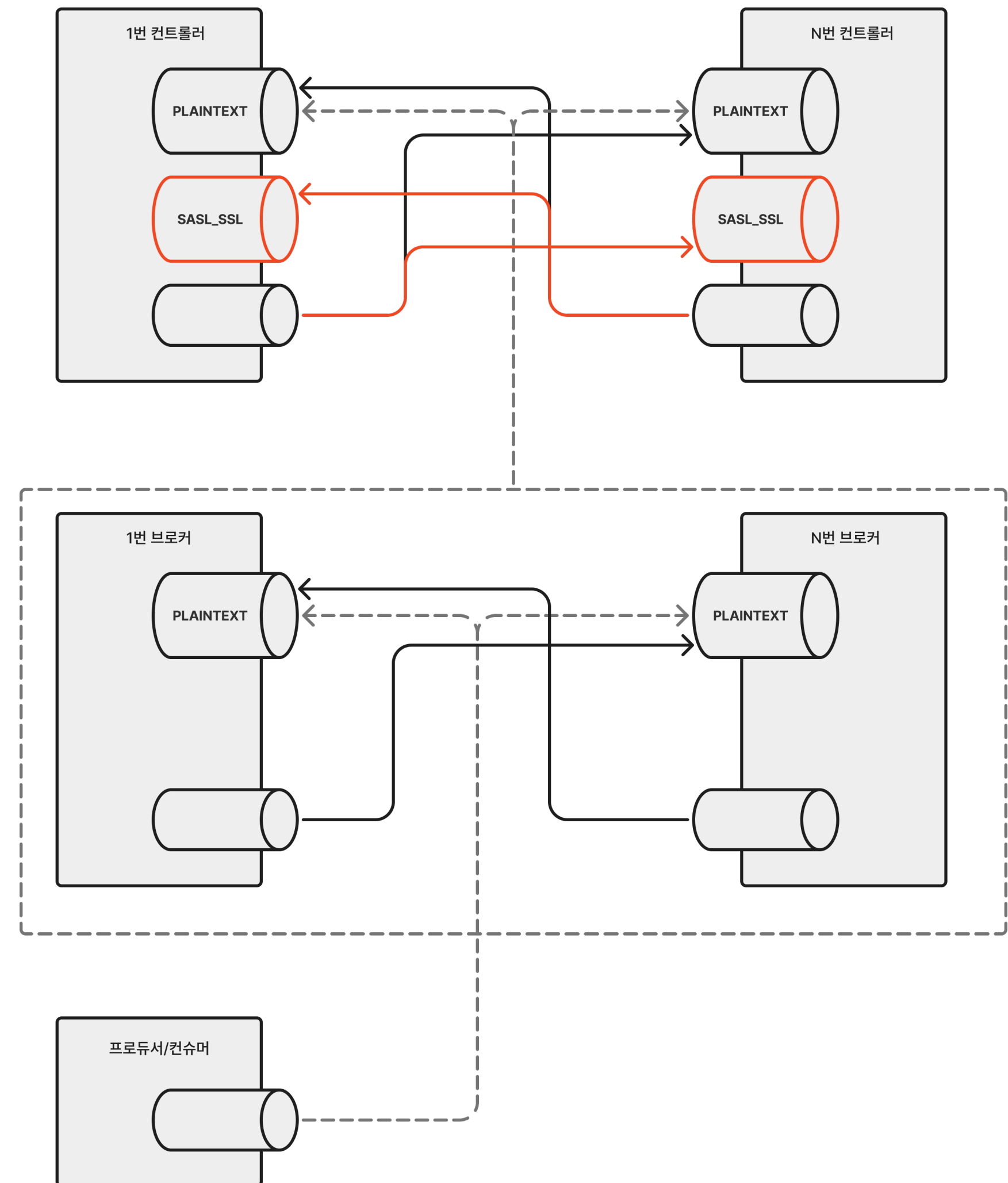
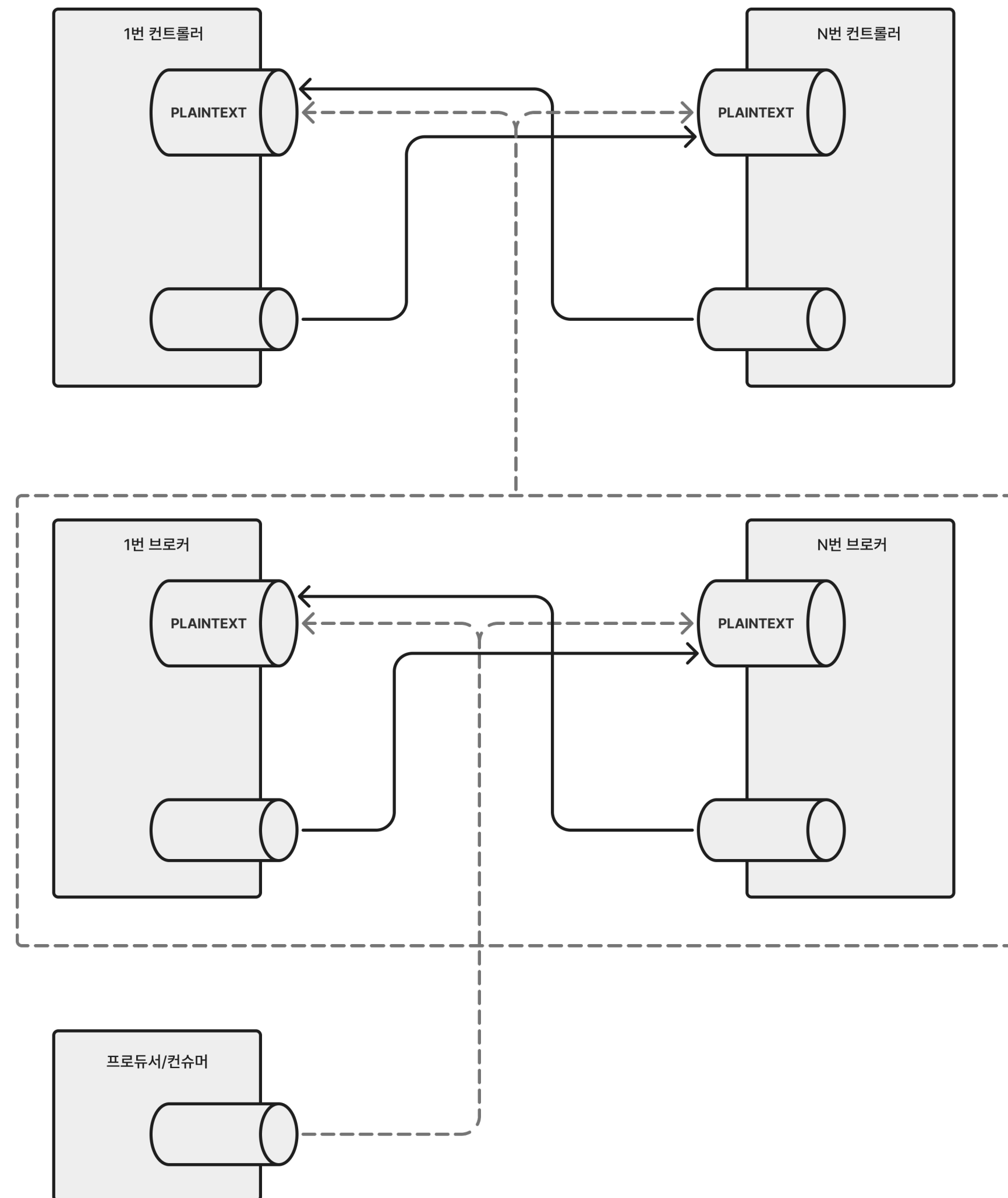
> kafka-features --bootstrap-server ${V3_9_BOOTSTRAP_SERVER} describe
Feature: kraft.version SupportedMinVersion: 0 SupportedMaxVersion: 1 FinalizedVersionLevel: 1 Epoch: 40551548
Feature: metadata.version      SupportedMinVersion: UNKNOWN 1 SupportedMaxVersion: 3.9-IV0      FinalizedVersionLevel: 3.9-IV0 Epoch: 40551548
```

1) <https://kafka.apache.org/42/operations/kraft/#static-versus-dynamic-kraft-quorums>

2) <https://kafka.apache.org/42/operations/kraft/#upgrade>

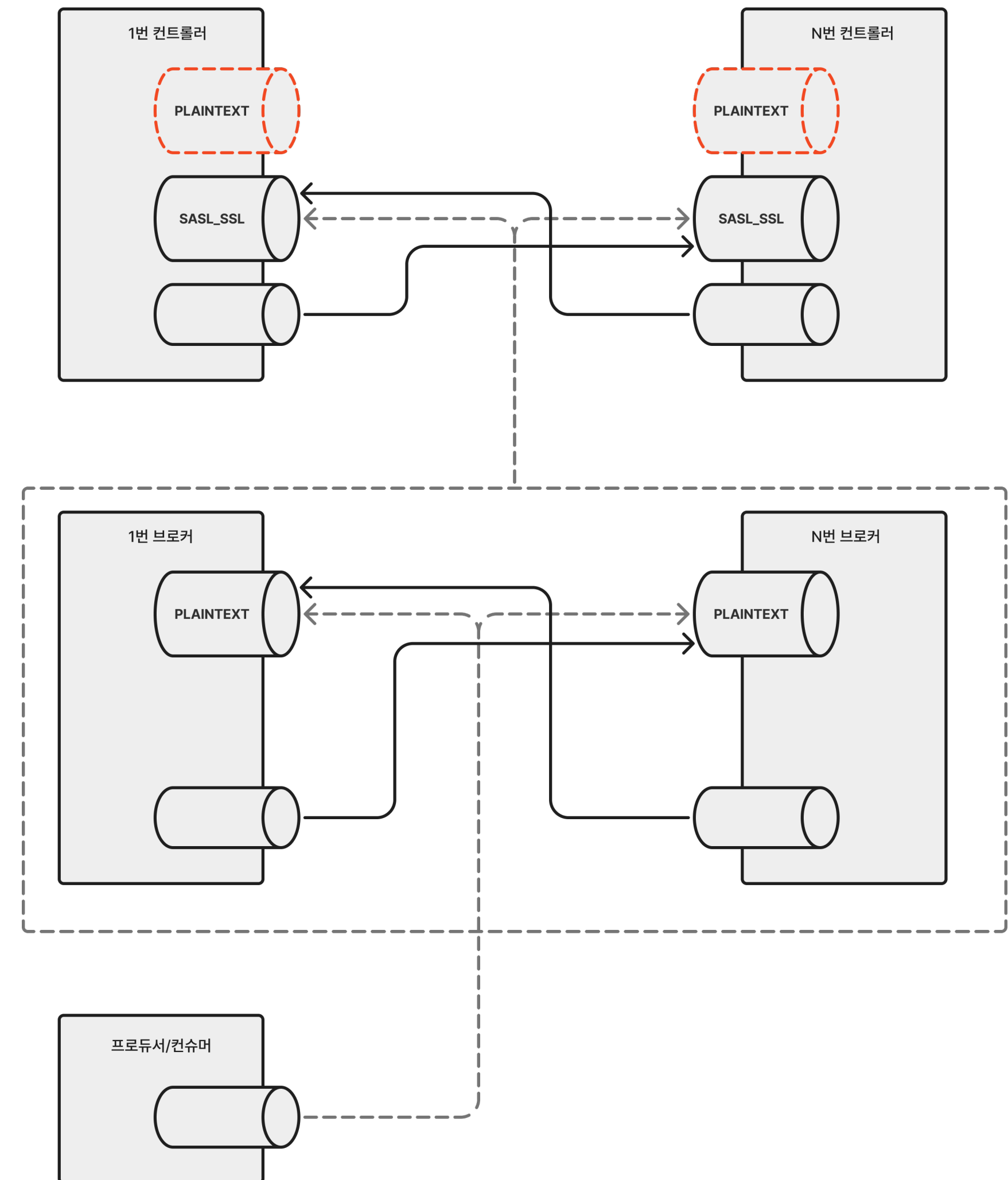
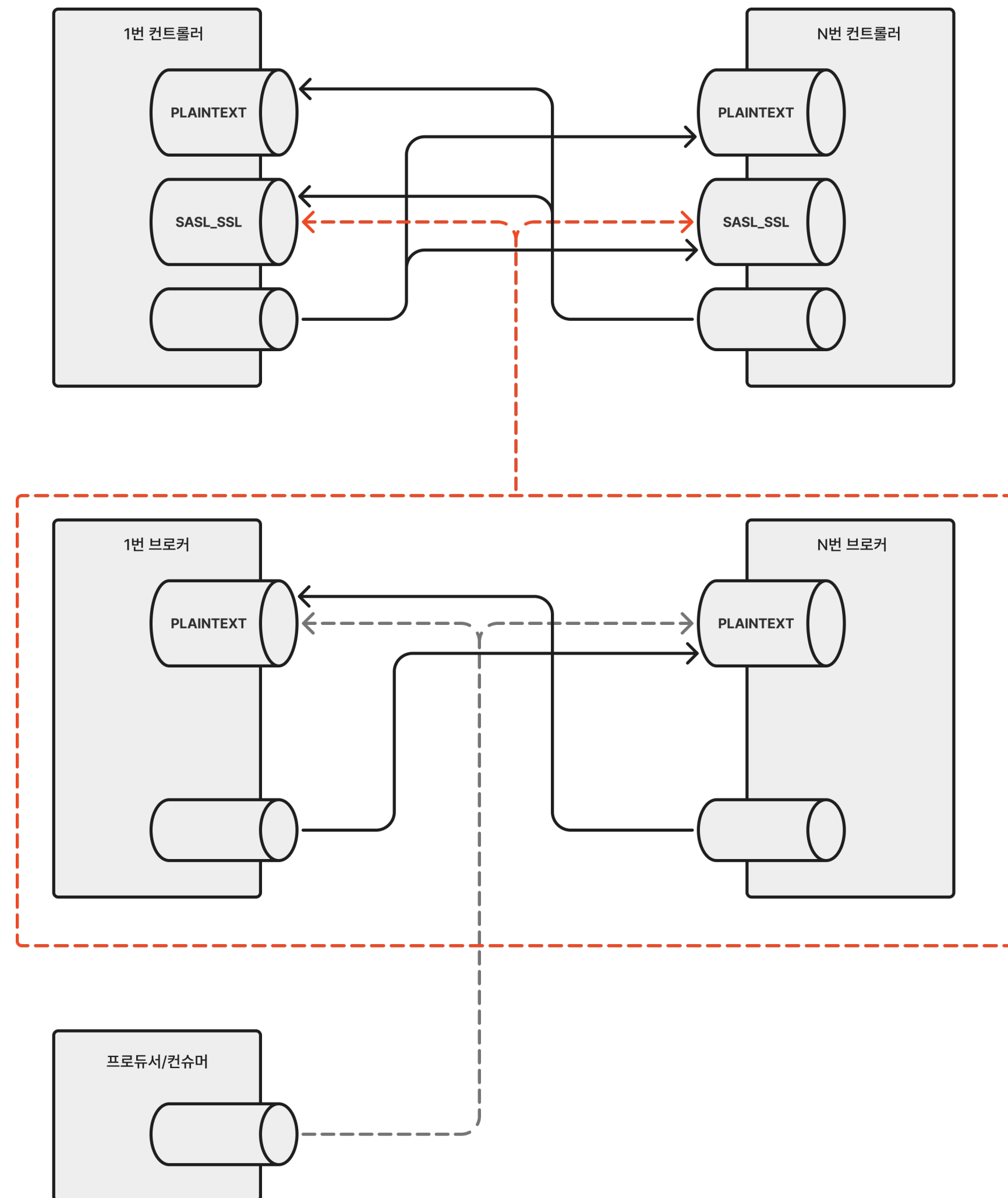
추가 고려 사항

PLAINTEXT → SASL_SSL 라이브 마이그레이션



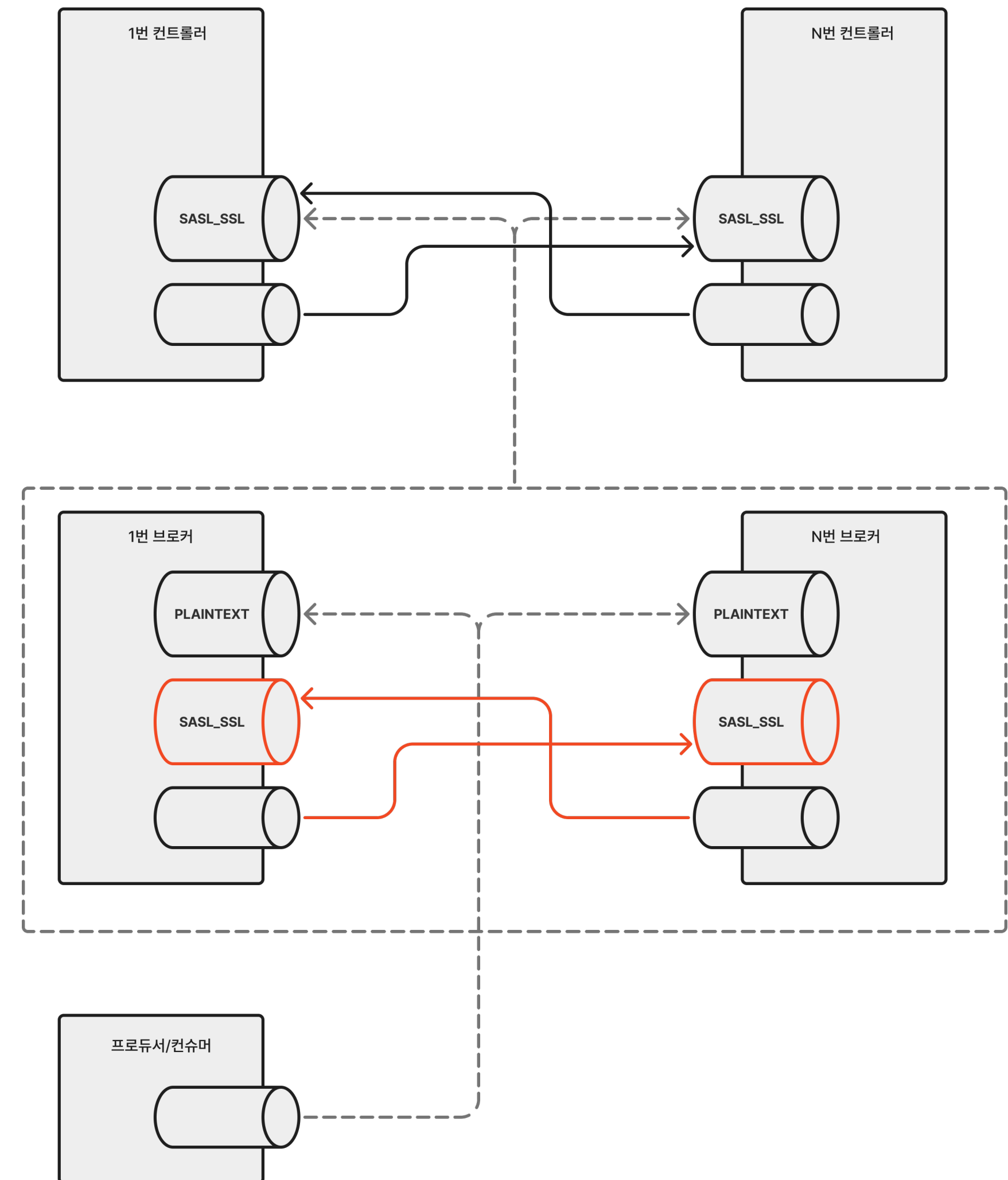
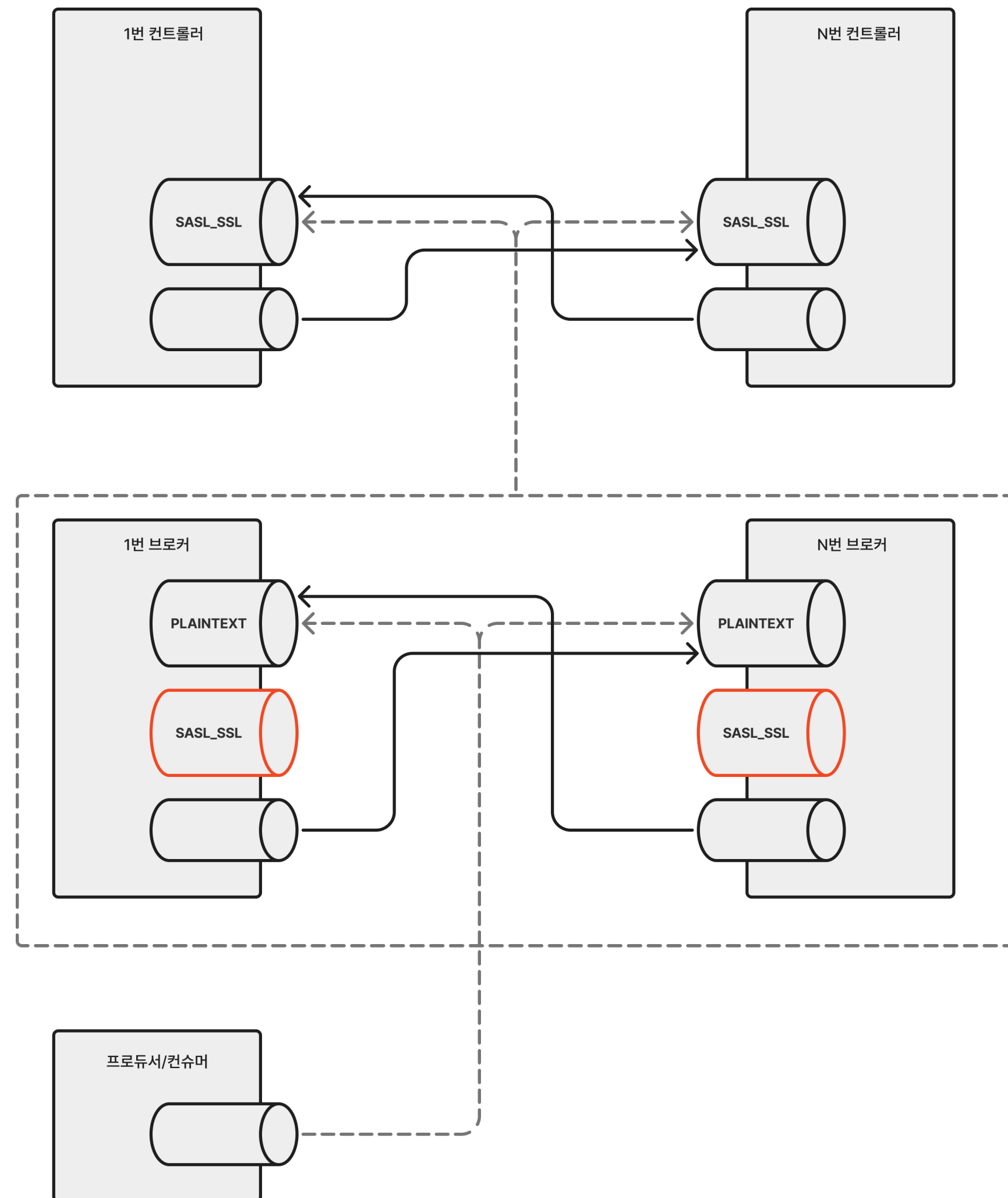
추가 고려 사항

PLAINTEXT → SASL_SSL 라이브 마이그레이션



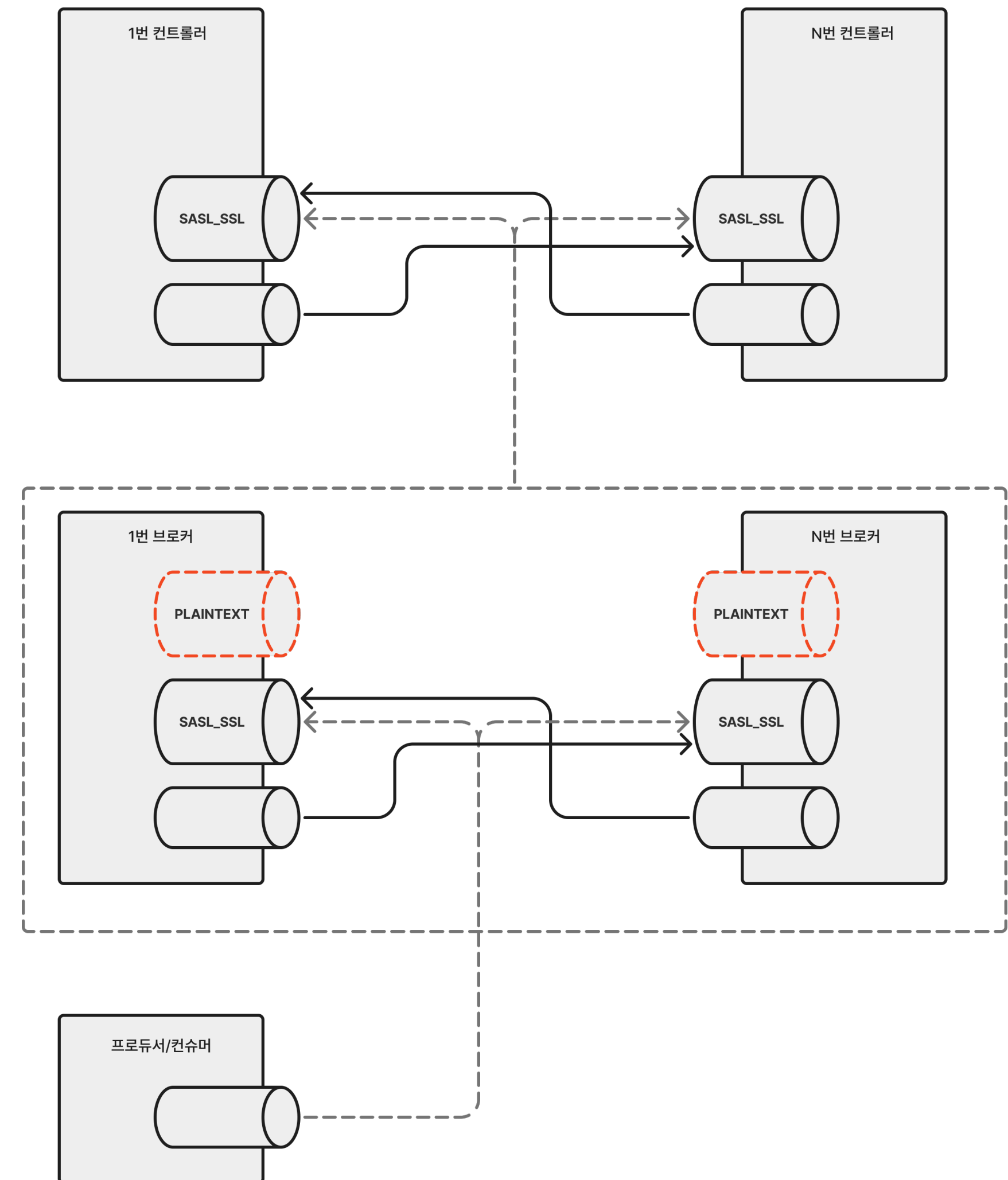
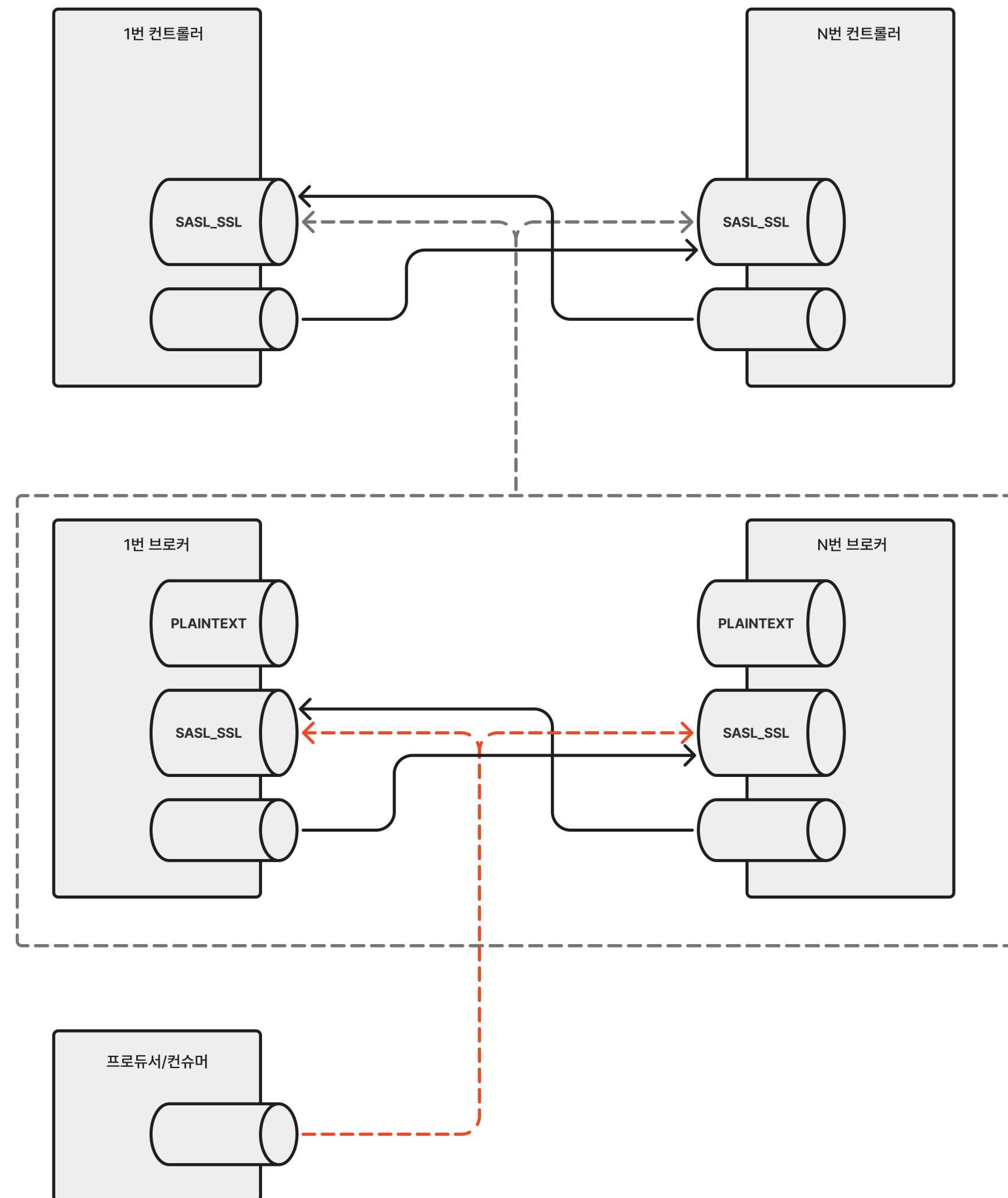
추가 고려 사항

PLAINTEXT → SASL_SSL 라이브 마이그레이션



추가 고려 사항

PLAINTEXT → SASL_SSL 라이브 마이그레이션



앞으로의 계획

Q&A